

DATABASE PROJECT REPORT

STUDENT INFORMATION

NAME: Cemre Bozdogan

STUDENT ID: GH1052353

COURSE: B103R Databases and Big Data (SS0326 Retake)

GITHUB LINK: <https://github.com/scarlettt01013-star/Hotel-Management-Database>

YOUTUBE VIDEO LINK: <https://youtu.be/EVc24g9pIWY>

INTRODUCTION

This project demonstrates the design and implementation of a relational database using SQL.

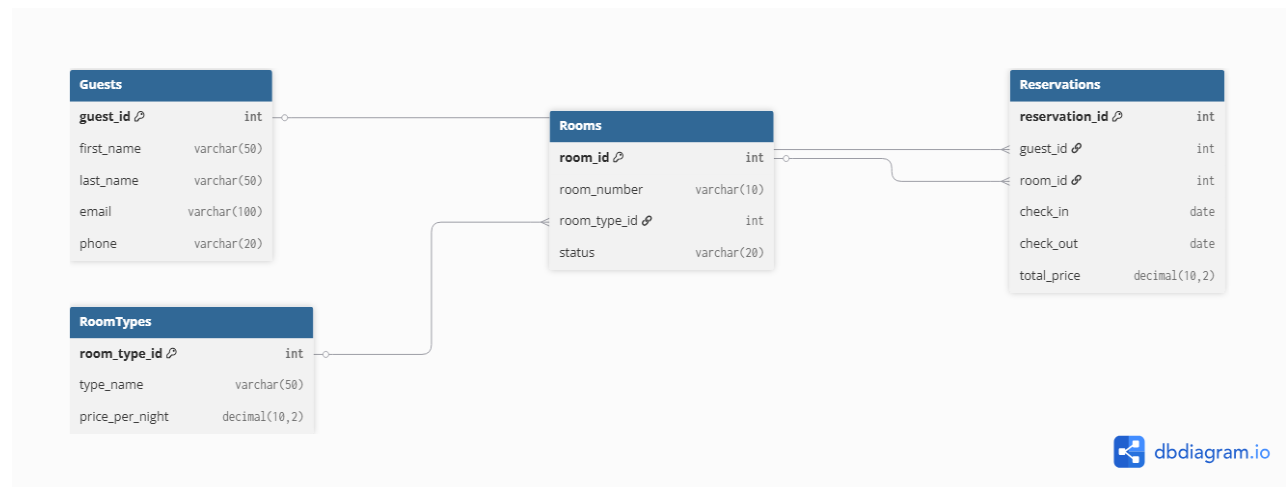
The database is developed for a **Hotel Reservation System** to efficiently manage hotel operations, including guest information, room types, rooms, and reservations. The system is designed to store and organize hotel data while maintaining relationships between different entities through primary and foreign keys.

The project also demonstrates various SQL operations such as **CREATE TABLE**, **INSERT**, **SELECT**, **JOIN**, **UPDATE**, **ORDER BY**, and **SUM** queries. These operations are used to manage hotel data, retrieve meaningful information, and generate reports related to reservations and room availability.

The main objective of this project is to demonstrate the principles of relational database design and show how SQL can be used to manage and analyze hotel reservation data efficiently.

SYSTEM DESIGN

The following Entity Relationship Diagram (ERD) illustrates the structure of the Hotel Reservation System database. It shows the main entities, their attributes, and the relationships between them. The database consists of four main tables: **Guests**, **RoomTypes**, **Rooms**, and **Reservations**. These tables are connected using primary keys and foreign keys to ensure data integrity and efficient management of hotel reservation information.



DATABASE CREATION:

```
CREATE DATABASE hotel_db;
```

```
USE hotel_db;
```

TABLE CREATION:

Guests Table:

```
CREATE TABLE Guests (  
    guest_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,
```

```
email VARCHAR(100),  
phone VARCHAR(20)  
);
```

RoomTypes Table:

```
CREATE TABLE RoomTypes (  
    room_type_id INT AUTO_INCREMENT PRIMARY KEY,  
    type_name VARCHAR(50),  
    price_per_night DECIMAL(10,2)  
);
```

Rooms Table:

```
CREATE TABLE Rooms (  
    room_id INT AUTO_INCREMENT PRIMARY KEY,  
    room_number VARCHAR(10),  
    room_type_id INT,  
    status VARCHAR(20),  
    FOREIGN KEY (room_type_id)  
        REFERENCES RoomTypes(room_type_id)  
);
```

Reservations Table:

```
CREATE TABLE Reservations (  
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
    guest_id INT,  
    room_id INT,  
    check_in DATE,  
    check_out DATE,
```

```
total_price DECIMAL(10,2),  
FOREIGN KEY (guest_id)  
REFERENCES Guests(guest_id),  
FOREIGN KEY (room_id)  
REFERENCES Rooms(room_id)  
);
```

INSERT DATA:

Guests:

```
INSERT INTO Guests (first_name, last_name, email, phone)  
VALUES  
( 'Ahmet', 'Yilmaz', 'ahmet@gmail.com', '5551111111'),  
( 'Ayse', 'Demir', 'ayse@gmail.com', '5552222222'),  
( 'Mehmet', 'Kaya', 'mehmet@gmail.com', '5553333333');
```

RoomTypes:

```
INSERT INTO RoomTypes (type_name, price_per_night)  
VALUES  
( 'Single', 80.00),  
( 'Double', 120.00),  
( 'Suite', 250.00);
```

Rooms:

```
INSERT INTO Rooms (room_number, room_type_id, status)  
VALUES  
( '101', 1, 'Available'),  
( '102', 2, 'Occupied'),
```

```
('201', 3, 'Available'),  
('202', 1, 'Available');
```

Reservations:

INSERT INTO Reservations

(guest_id, room_id, check_in, check_out, total_price)

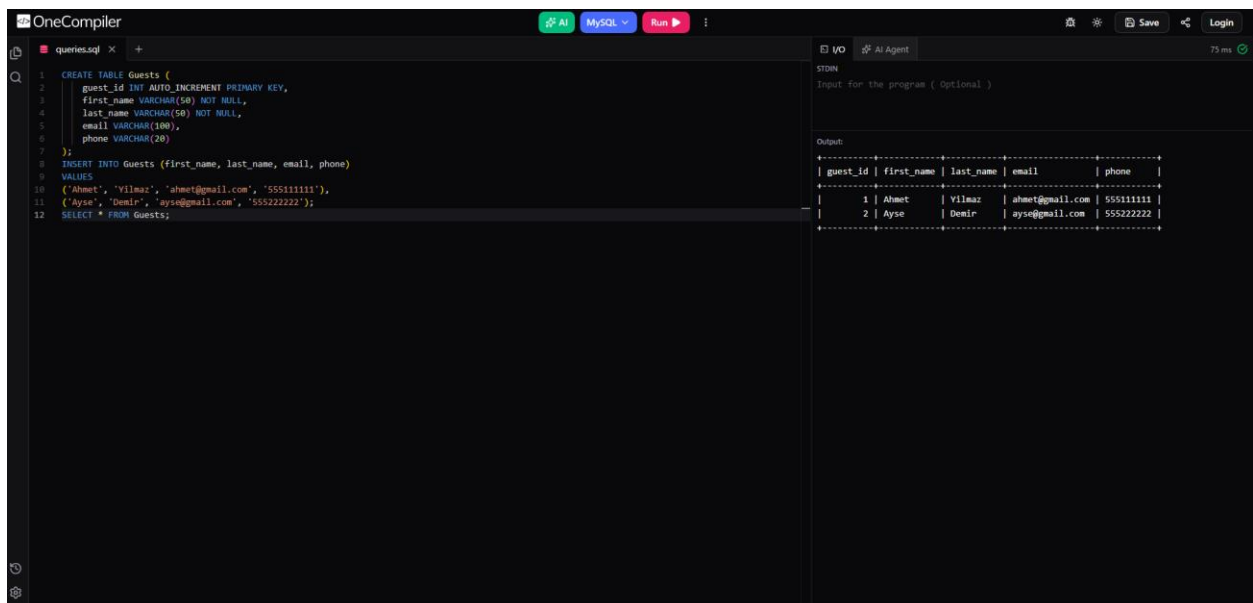
VALUES

```
(1, 1, '2025-07-01', '2025-07-05', 320.00),  
(2, 2, '2025-07-10', '2025-07-12', 240.00),  
(3, 3, '2025-07-15', '2025-07-18', 750.00);
```

QUERIS:

Display All Guests

SELECT * FROM Guests;



The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

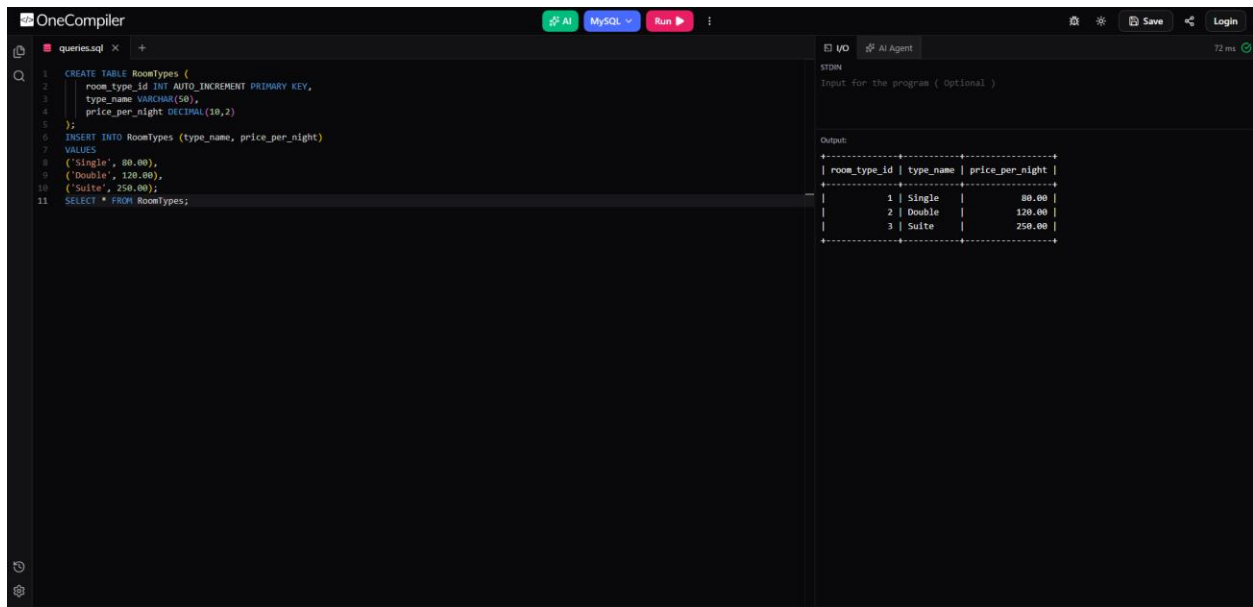
```
1 CREATE TABLE Guests (  
2   guest_id INT AUTO_INCREMENT PRIMARY KEY,  
3   first_name VARCHAR(50) NOT NULL,  
4   last_name VARCHAR(50) NOT NULL,  
5   email VARCHAR(100),  
6   phone VARCHAR(20)  
7 );  
8 INSERT INTO Guests (first_name, last_name, email, phone)  
9 VALUES  
10 ('Ahmet', 'Yilmaz', 'ahmet@gmail.com', '5551111111'),  
11 ('Ayse', 'Demir', 'ayse@gmail.com', '5552222222');  
12 SELECT * FROM Guests;
```

The right pane shows the output of the query, displaying a table with 5 columns: guest_id, first_name, last_name, email, and phone. The table contains 2 rows of data.

guest_id	first_name	last_name	email	phone
1	Ahmet	Yilmaz	ahmet@gmail.com	5551111111
2	Ayşe	Demir	ayse@gmail.com	5552222222

Display All Rooms

SELECT * FROM Rooms;



Show Reservations with Guest Information

SELECT

g.first_name,
g.last_name,
r.room_number,
res.check_in,
res.check_out,
res.total_price

FROM Reservations res

JOIN Guests g

ON res.guest_id = g.guest_id

JOIN Rooms r

ON res.room_id = r.room_id;

queries.sql

```
1 CREATE TABLE Reservations (
2   reservation_id INT AUTO_INCREMENT PRIMARY KEY,
3   guest_id INT,
4   room_id INT,
5   check_in DATE,
6   check_out DATE,
7   total_price DECIMAL(10,2)
8 );
9 INSERT INTO Reservations
10 (guest_id, room_id, check_in, check_out, total_price)
11 VALUES
12 (1, 1, '2025-07-01', '2025-07-05', 320.00),
13 (2, 2, '2025-07-10', '2025-07-12', 240.00);
14 SELECT * FROM Reservations;
```

VO AI Agent 90 ms

SQL

Input for the program (Optional)

Output:

reservation_id	guest_id	room_id	check_in	check_out	total_price
1	1	1	2025-07-01	2025-07-05	320.00
2	2	2	2025-07-10	2025-07-12	240.00

Available Rooms

SELECT *

FROM Rooms

WHERE status = 'Available';

queries.sql

```
61 SELECT *
62 FROM Rooms
63 WHERE status='Available';
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
```

VO AI Agent 96 ms

SQL

Input for the program (Optional)

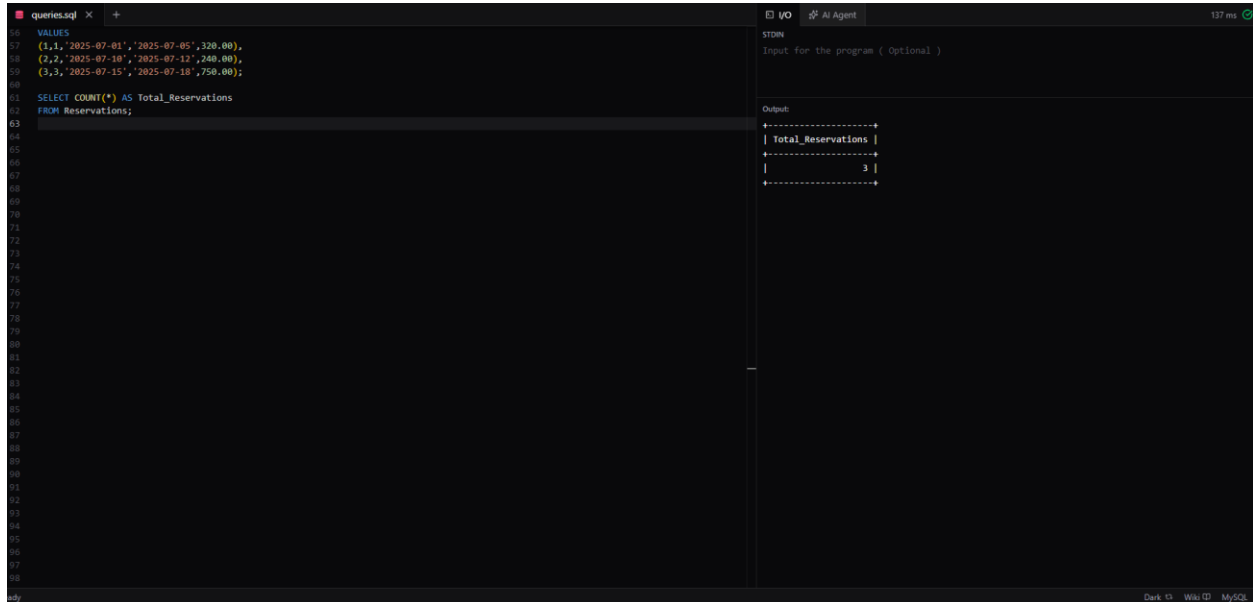
Output:

room_id	room_number	room_type_id	status
1	101	1	Available
3	201	3	Available

Dark Wikis MySQL

Total Reservations

```
SELECT COUNT(*) AS Total_Reservations  
FROM Reservations;
```



The screenshot shows a MySQL IDE interface. On the left, a query editor window titled 'queries.sql' contains the following SQL code:

```
VALUES  
(1,1,'2025-07-01','2025-07-05',320.00),  
(2,2,'2025-07-10','2025-07-12',240.00),  
(3,3,'2025-07-15','2025-07-18',750.00);  
  
SELECT COUNT(*) AS Total_Reservations  
FROM Reservations;
```

On the right, the 'Output' tab is active, displaying the result of the query. The output is a single row with the value 3, representing the total number of reservations.

Total_Reservations
3

Reservation Dates

```
SELECT  
reservation_id,  
check_in,  
check_out  
FROM Reservations;
```



```
queries.sql x +
VALUES
(1,1,'2025-07-01','2025-07-05',320.00),
(2,2,'2025-07-10','2025-07-12',240.00),
(3,3,'2025-07-15','2025-07-18',750.00);
SELECT reservation_id, check_in, check_out
FROM Reservations;
```

113 ms

STOP

Input for the program (Optional)

Output:

reservation_id	check_in	check_out
1	2025-07-01	2025-07-05
2	2025-07-10	2025-07-12
3	2025-07-15	2025-07-18

Dark 13 Wiki ID MySQL

Before Update

SELECT * FROM Rooms

WHERE room_id = 1;

```
queries.sql x +
(guest_id, room_id, check_in, check_out, total_price)
VALUES
(1,1,'2025-07-01','2025-07-05',320.00),
(2,2,'2025-07-10','2025-07-12',240.00),
(3,3,'2025-07-15','2025-07-18',750.00);
SELECT * FROM Rooms
WHERE room_id = 1;
```

136 ms

STOP

Input for the program (Optional)

Output:

room_id	room number	room_type_id	status
1	101	1	Available

Dark 13 Wiki ID MySQL

Update

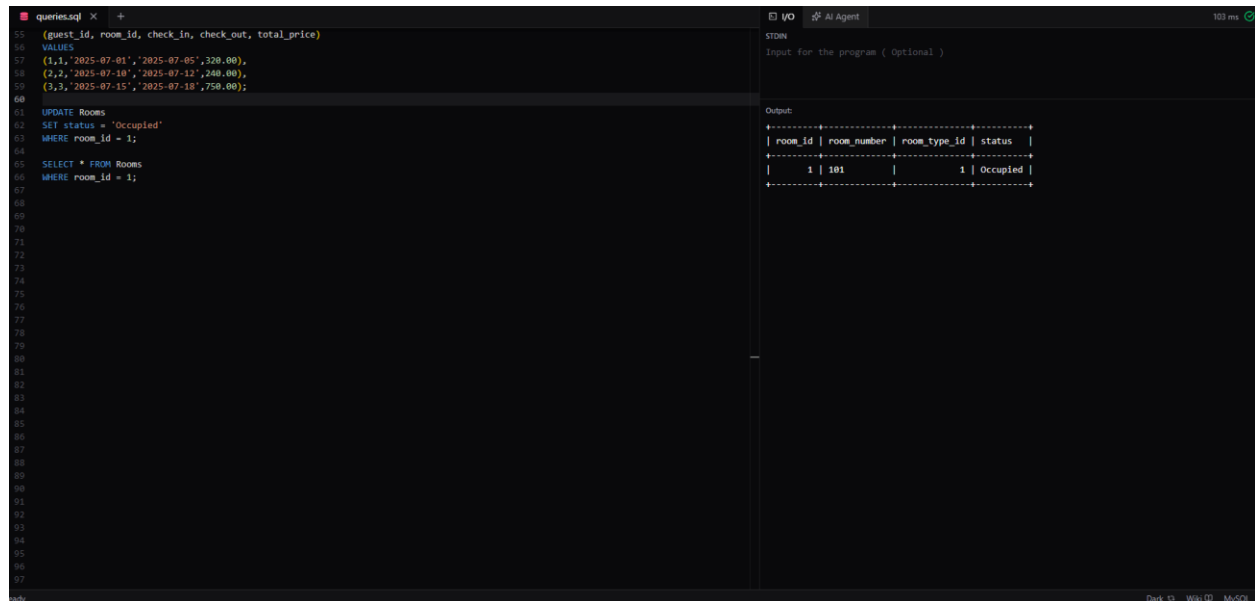
UPDATE Rooms

SET status = 'Occupied'

WHERE room_id = 1;

SELECT * FROM Rooms

WHERE room_id = 1;



The screenshot shows a SQL IDE with a query editor on the left and a results/output pane on the right. The query editor contains the following SQL code:

```
55 (guest_id, room_id, check_in, check_out, total_price)
56 VALUES
57 (1,1,'2025-07-01','2025-07-05',320.00),
58 (2,2,'2025-07-10','2025-07-12',240.00),
59 (3,3,'2025-07-15','2025-07-18',750.00);
60
61 UPDATE Rooms
62 SET status = 'Occupied'
63 WHERE room_id = 1;
64
65 SELECT * FROM Rooms
66 WHERE room_id = 1;
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
```

The results/output pane on the right shows the output of the query. It includes a section for the 'Output' of the SELECT statement, which displays a table with the following data:

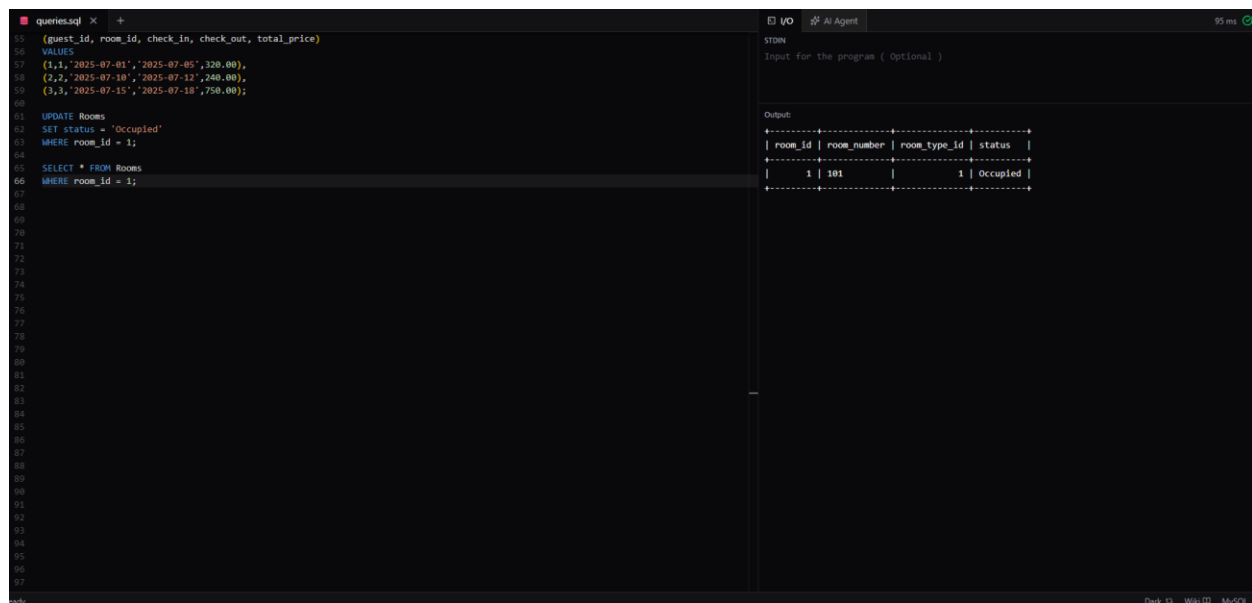
room_id	room_number	room_type_id	status
1	101	1	Occupied

The output pane also shows a section for the 'STDIN' input, which is empty. The output pane is titled 'Output' and has a '103 ms' execution time indicator.

After Update

SELECT * FROM Rooms

WHERE room_id = 1;



The screenshot shows a MySQL query editor with a dark theme. The left pane contains SQL code for inserting reservation data, updating room status, and selecting all records from the Rooms table. The right pane shows the output of the last query, which is a table with four columns: room_id, room_number, room_type_id, and status. The output shows one row with room_id 1, room_number 101, room_type_id 1, and status Occupied.

```
55 (guest_id, room_id, check_in, check_out, total_price)
56 VALUES
57 (1,1,'2025-07-01','2025-07-05',320.00),
58 (2,2,'2025-07-10','2025-07-12',240.00),
59 (3,3,'2025-07-15','2025-07-18',750.00);
60
61 UPDATE Rooms
62 SET status = 'Occupied'
63 WHERE room_id = 1;
64
65 SELECT * FROM Rooms
66 WHERE room_id = 1;
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
```

Output:

room_id	room_number	room_type_id	status
1	101	1	Occupied

Conclusion

This project successfully developed a **Hotel Management Database System** using MySQL. The database was designed to manage important hotel information, including guests, room types, rooms, and reservations. By organizing the data into separate but related tables, the system provides an efficient way to store and retrieve information while reducing data duplication.

During the project, SQL commands such as **CREATE TABLE**, **INSERT INTO**, and **SELECT** were used to create the database, add sample records, and perform different queries. These operations demonstrated how a relational database can simplify hotel management tasks, such as checking room availability and managing reservations.

Overall, this project provided a practical understanding of database design and SQL. It also showed how a well-structured database can improve the efficiency and accuracy of hotel operations. In the future, the system can be expanded with additional features such as payment management, staff information, and online booking to create a more complete hotel management solution.

Challenges

During the development of this project, several challenges were encountered. One of the main difficulties was designing the database structure and creating proper relationships between tables.

It was important to organize the data correctly so that guests, room types, rooms, and reservations were connected in a logical and efficient way.

Another challenge was writing SQL queries without syntax errors. Small mistakes in table names, column names, or query syntax sometimes caused execution errors, which required careful debugging and testing. It was also important to insert data in the correct order and ensure that all tables contained valid information before running queries.

Implementing the **UPDATE** operation was another challenge. After updating room information, it was necessary to verify that the changes were successfully applied by running additional **SELECT** queries. This helped ensure that the database was updated correctly and maintained accurate records.

Despite these challenges, all issues were solved through testing and debugging. The final database successfully supports creating tables, inserting data, updating records, and retrieving information using SQL queries, providing a functional Hotel Management Database System.